

最长连续子序列

题目描述

有N个正整数组成的一个序列。给定整数sum，求长度最长的连续 **子序列**，使他们的和等于sum，返回此子序列的长度，如果没有满足要求的序列，返回-1。

输入描述

第一行输入是：N个正整数组成的一个序列
第二行输入是：给定整数sum

输出描述

最长的连续子序列的长度

备注

- 输入序列仅由数字和英文逗号构成，数字之间采用英文逗号分隔
- 序列长度：1 <= N <= 200
- 输入序列不考虑异常情况

用例

输入	1,2,3,4,2 6
----	----------------

输出	3
说明	1,2,3和4,2两个序列均能满足要求，所以最长的连续序列为1,2,3，因此结果为3。

输入	1,2,3,4,2 20
输出	-1
说明	没有满足要求的子序列，返回-1

前缀和解法

本题数量级不大，可以考虑求解输入序列的前缀和数组，实现O(1) **时间复杂度计算** 任意区间和，关于前缀和可以看：[算法设计 - 前缀和 & 差分数列_伏城之外的博客-CSDN博客](#)

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const nums = (await readline()).split(",").map(Number);
7      const target = parseInt(await readline());
8
9      console.log(getResult(nums, target));
10 })();
11
12 function getResult(nums, target) {
13     const n = nums.length;
14
15     const preSum = new Array(n + 1).fill(0);
```

运行

```

16 |   for (let i = 1; i <= n; i++) {
17 |       preSum[i] = preSum[i - 1] + nums[i - 1];
18 |   }
19 |
20 |   let maxLen = -1;
21 |   for (let l = 1; l <= n; l++) {
22 |       for (let r = l; r <= n; r++) {
23 |           if (preSum[r] - preSum[l - 1] == target) {
24 |               maxLen = Math.max(maxLen, r - l + 1);
25 |           }
26 |       }
27 |   }
28 |
29 |   return maxLen;
30 | }

```

Java算法源码

```

1 | import java.util.Arrays;
2 | import java.util.Scanner;
3 |
4 | public class Main {
5 |     public static void main(String[] args) {
6 |         Scanner sc = new Scanner(System.in);
7 |
8 |         int[] nums = Arrays.stream(sc.nextLine().split(",")).mapToInt(Integer::parseInt).toArray();
9 |         int target = Integer.parseInt(sc.nextLine());
10 |
11 |         System.out.println(getResult(nums, target));
12 |     }
13 |
14 |     public static int getResult(int[] nums, int target) {
15 |         int n = nums.length;
16 |

```

```

17 // 前缀和数组
18     int[] preSum = new int[n + 1];
19     for (int i = 1; i <= n; i++) {
20         preSum[i] = preSum[i - 1] + nums[i - 1];
21     }
22
23 // 记录最长连续子序列的长度
24     int maxLen = -1;
25
26 // 求解所有连续子序列的区间和
27     for (int l = 1; l <= n; l++) {
28         for (int r = l; r <= n; r++) {
29             if (preSum[r] - preSum[l - 1] == target) {
30                 maxLen = Math.max(maxLen, r - l + 1);
31             }
32         }
33     }
34
35     return maxLen;
36 }
37 }

```

Python算法源码

```

1 # 输入获取
2 nums = list(map(int, input().split(",")))
3 target = int(input())
4
5
6 # 算法入口
7 def getResult():
8     n = len(nums)
9
10     preSum = [0] * (n + 1)
11     for i in range(1, n + 1):

```

```

12         preSum[i] = preSum[i - 1] + nums[i - 1]
13     }
14     maxLen = -1
15
16     for l in range(1, n + 1):
17         for r in range(1, n + 1):
18             if preSum[r] - preSum[l - 1] == target:
19                 maxLen = max(maxLen, r - l + 1)
20
21     return maxLen
22
23
24 # 算法调用
25 print(getResult())

```

C算法源码

```

1  #include <stdio.h>
2
3  #define MAX(a, b) (a) > (b) ? (a) : (b)
4
5  #define MAX_SIZE 200
6
7  int getResult(int nums[], int nums_size, int target);
8
9  int main() {
10     int nums[MAX_SIZE];
11     int nums_size = 0;
12
13     while (scanf("%d", &nums[nums_size++])) {
14         if (getchar() != ',') break;
15     }
16
17     int sum;
18     scanf("%d", &sum);

```

```

19 |     int preSum[nums_size + 1];
20 |
21 |
22 |     preSum[0] = 0;
23 |     for (int i = 1; i <= nums_size; i++) {
24 |         preSum[i] = preSum[i - 1] + nums[i - 1];
25 |     }
26 |
27 |     int maxLen = -1;
28 |     for (int l = 1; l <= nums_size; l++) {
29 |         for (int r = l; r <= nums_size; r++) {
30 |             if (preSum[r] - preSum[l - 1] == sum) {
31 |                 maxLen = MAX(maxLen, r - l + 1);
32 |             }
33 |         }
34 |     }
35 |
36 |     printf("%d\n", maxLen);
37 |
38 |     return 0;
39 | }

```

C++ 算法源码

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | vector<int> splitCin(char delimiter) {
6 |     string s;
7 |     cin >> s;
8 |
9 |     stringstream ss(s);
10 |     string token;
11 |

```

运行

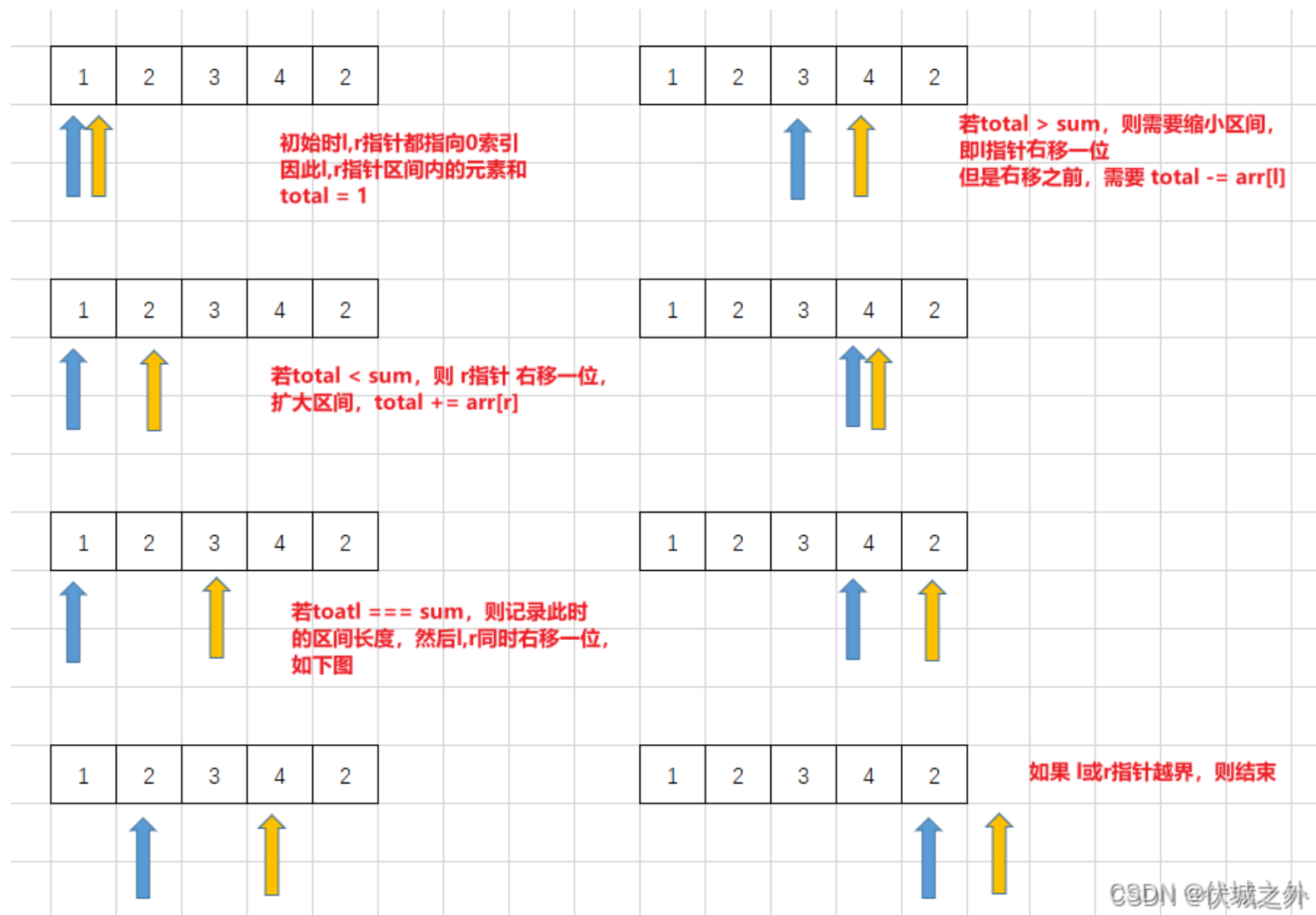
```

12     vector<int> res;
13     while (getline(ss, token, delimiter)) {
14         res.emplace_back(stoi(token));
15     }
16
17     return res;
18 }
19
20
21 int main() {
22     vector<int> nums = splitCin(',');
23
24     int sum;
25     cin >> sum;
26
27     vector<int> preSum(nums.size() + 1, 0);
28     for (int i = 1; i <= nums.size(); i++) {
29         preSum[i] = preSum[i - 1] + nums[i - 1];
30     }
31
32     int maxLen = -1;
33
34     for (int i = 1; i <= nums.size(); i++) {
35         for (int j = i; j <= nums.size(); j++) {
36             if (preSum[j] - preSum[i - 1] == sum) {
37                 maxLen = max(maxLen, j - i + 1);
38             }
39         }
40     }
41
42     cout << maxLen << endl;
43
44     return 0;
45 }

```

滑动窗口解法

本题还可以利用滑动窗口解决。



同时, 有一种特殊情况



2023.06.12

本题的第二行输入是: 给定整数sum

因此sum可能取负数, 0, 正数, 对于sum为负数, 0的情况, 可以直接返回-1, 因为第一行输入是: N个正整数组成的一个序列

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const nums = (await readline()).split(",").map(Number);
7      const sum = parseInt(await readline());
8
9      let l = 0;
10     let r = 0;
11     let window_sum = 0;
```

```

12 |
13 |     let maxLen = 0;
14 |
15 |     while (r < nums.length) {
16 |         if (l > r || window_sum + nums[r] <= sum) {
17 |             window_sum += nums[r++];
18 |         } else {
19 |             window_sum -= nums[l++];
20 |         }
21 |
22 |         if (window_sum == sum) {
23 |             maxLen = Math.max(maxLen, r - l);
24 |         }
25 |     }
26 |
27 |     console.log(maxLen == 0 ? -1 : maxLen);
28 | })();

```

Java算法源码

```

1 | import java.util.Arrays;
2 | import java.util.Scanner;
3 |
4 | public class Main {
5 |     public static void main(String[] args) {
6 |         Scanner sc = new Scanner(System.in);
7 |
8 |         int[] nums = Arrays.stream(sc.nextLine().split(",")).mapToInt(Integer::parseInt).toArray();
9 |         int sum = Integer.parseInt(sc.nextLine());
10 |
11 |         int l = 0;
12 |         int r = 0;
13 |         int window_sum = 0;
14 |
15 |         int maxLen = 0;

```

```

16 |
17 |         while (r < nums.length) {
18 |             if (l > r || window_sum + nums[r] <= sum) {
19 |                 window_sum += nums[r++];
20 |             } else {
21 |                 window_sum -= nums[l++];
22 |             }
23 |
24 |             if (window_sum == sum) {
25 |                 maxLen = Math.max(maxLen, r - l);
26 |             }
27 |         }
28 |
29 |         System.out.println(maxLen == 0 ? -1 : maxLen);
30 |     }
31 | }

```

Python算法源码

```

1 | # 输入获取
2 | nums = list(map(int, input().split(",")))
3 | total = int(input())
4 |
5 |
6 | # 算法入口
7 | def getResult():
8 |     l = 0
9 |     r = 0
10 |     window_sum = 0
11 |
12 |     maxLen = 0
13 |
14 |     while r < len(nums):
15 |         if l > r or window_sum + nums[r] <= total:
16 |             window_sum += nums[r]

```

```

17         r += 1 | else:
18         window_sum -= nums[l]
19         l += 1
20
21
22     if window_sum == total:
23         maxLen = max(maxLen, r - l)
24
25     return -1 if maxLen == 0 else maxLen
26
27
28 # 算法调用
29 print(getResult())

```

C算法源码

```

1  #include <stdio.h>
2  #include <math.h>
3
4  #define MAX_SIZE 200
5
6  int main() {
7      int nums[MAX_SIZE];
8      int nums_size = 0;
9
10     while (scanf("%d", &nums[nums_size++])) {
11         if (getchar() != ',') break;
12     }
13
14     int sum;
15     scanf("%d", &sum);
16
17     int l = 0;
18     int r = 0;

```

```

19     int window_sum = 0;
20
21     int maxLen = 0;
22
23     while (r < nums_size) {
24         if (l > r || window_sum + nums[r] <= sum) {
25             window_sum += nums[r++];
26         } else {
27             window_sum -= nums[l++];
28         }
29
30         if (window_sum == sum) {
31             maxLen = (int) fmax(maxLen, r - l);
32         }
33     }
34
35     printf("%d\n", maxLen == 0 ? -1 : maxLen);
36
37     return 0;
38 }

```

C++ 算法源码

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  vector<int> splitCin(char delimiter) {
6      string s;
7      cin >> s;
8
9      stringstream ss(s);
10     string token;
11
12     vector<int> res;

```

```
13 | 14 |     while (getline(ss, token, delimiter)) {
15 |     res.emplace_back(stoi(token));
16 |     }
17 |
18 |     return res;
19 | }
20 |
21 | int main() {
22 |     vector<int> nums = splitCin(',');
23 |
24 |     int sum;
25 |     cin >> sum;
26 |
27 |     int l = 0;
28 |     int r = 0;
29 |     int window_sum = 0;
30 |
31 |     int maxLen = 0;
32 |
33 |     while (r < nums.size()) {
34 |         if (l > r || window_sum + nums[r] <= sum) {
35 |             window_sum += nums[r++];
36 |         } else {
37 |             window_sum -= nums[l++];
38 |         }
39 |
40 |         if (window_sum == sum) {
41 |             maxLen = max(maxLen, r - l);
42 |         }
43 |     }
44 |
45 |     cout << (maxLen == 0 ? -1 : maxLen) << endl;
46 |
47 |     return 0;
48 | }
```

